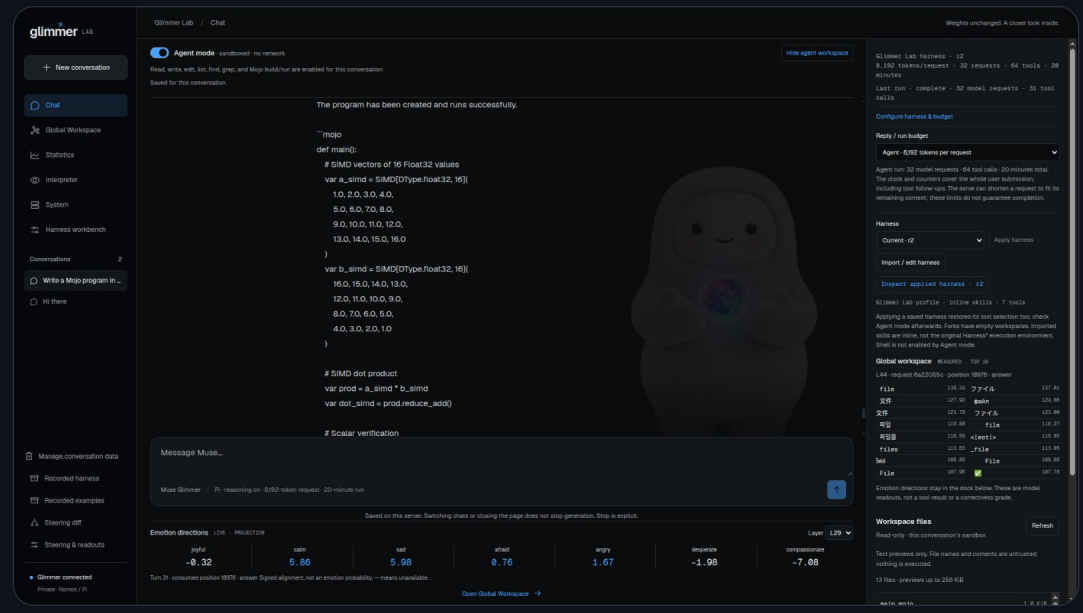
Glimmer Lab

See inside a local agent while it works, then make it better without training.

Adam Kruger · Light of Baldr LLC

Meta Global AI Developer Hackathon 2026 · Indie track



THE PROJECT

Why I built it

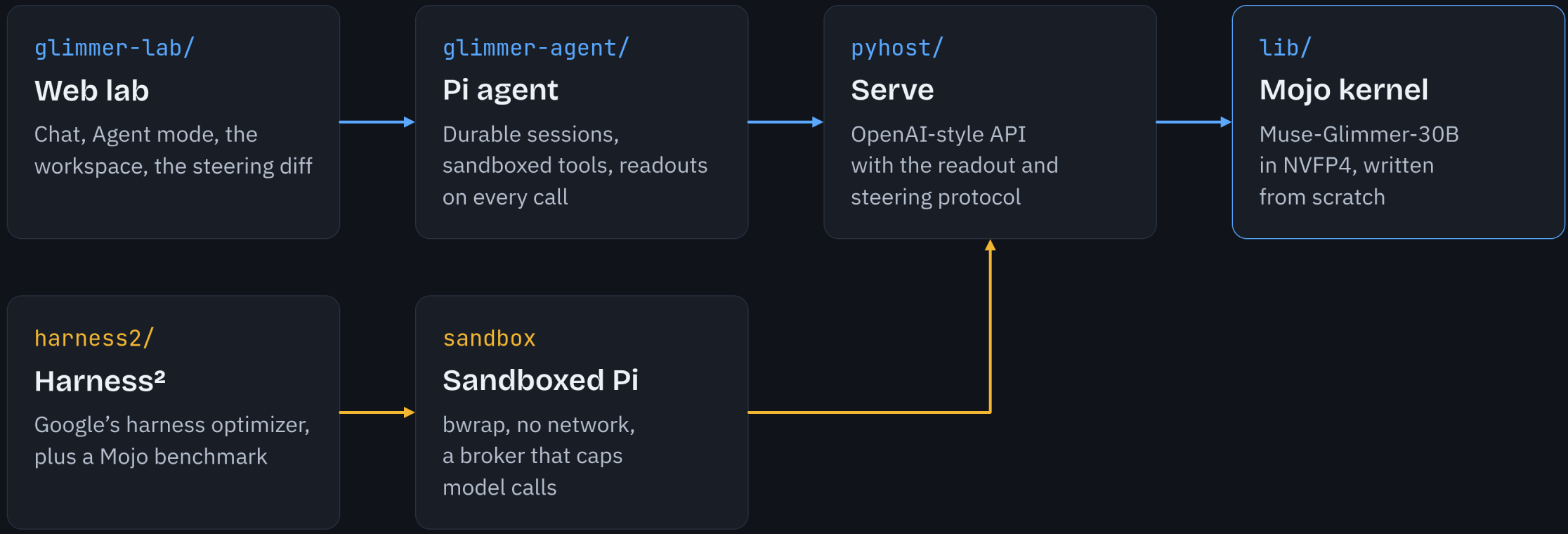
> “I currently manage a fleet of agents who help me with research and development, my day job, and managing operations. [...] With that said, it’s not perfect. Cloud credits and subscriptions to the various frontier labs can be expensive. And there are a lot of tasks, day-to-day tasks for example, or even experiments, that feel like frontier intelligence is wasted on them. So for this hackathon, what I wanted to build wasn’t another agentic system. [...] What I wanted to build was a system that helps me understand where the cracks are, the shortcomings in local AI, and of course in this case that’s Glimmer 30B, [...] a system that is reliable and trustworthy.”

Adam Kruger, from the demo rehearsal

WHAT IT DOES

Four parts, one repository

Glimmer, its kernel and the readouts run on one 24 GB Blackwell GPU.



WHAT IT DOES

A kernel written from scratch in Mojo

Glimmer on one 24 GB GPU

Muse-Glimmer-30B in NVFP4 with W4A4, a 32K context and speculative decoding, on an RTX 5090 laptop GPU.

Readouts inside the kernel

At every generated token: the emotion readout and the Global Workspace, read from the residual stream.

Steering

Add a direction to the residual stream at a chosen layer, for the response or the whole context.

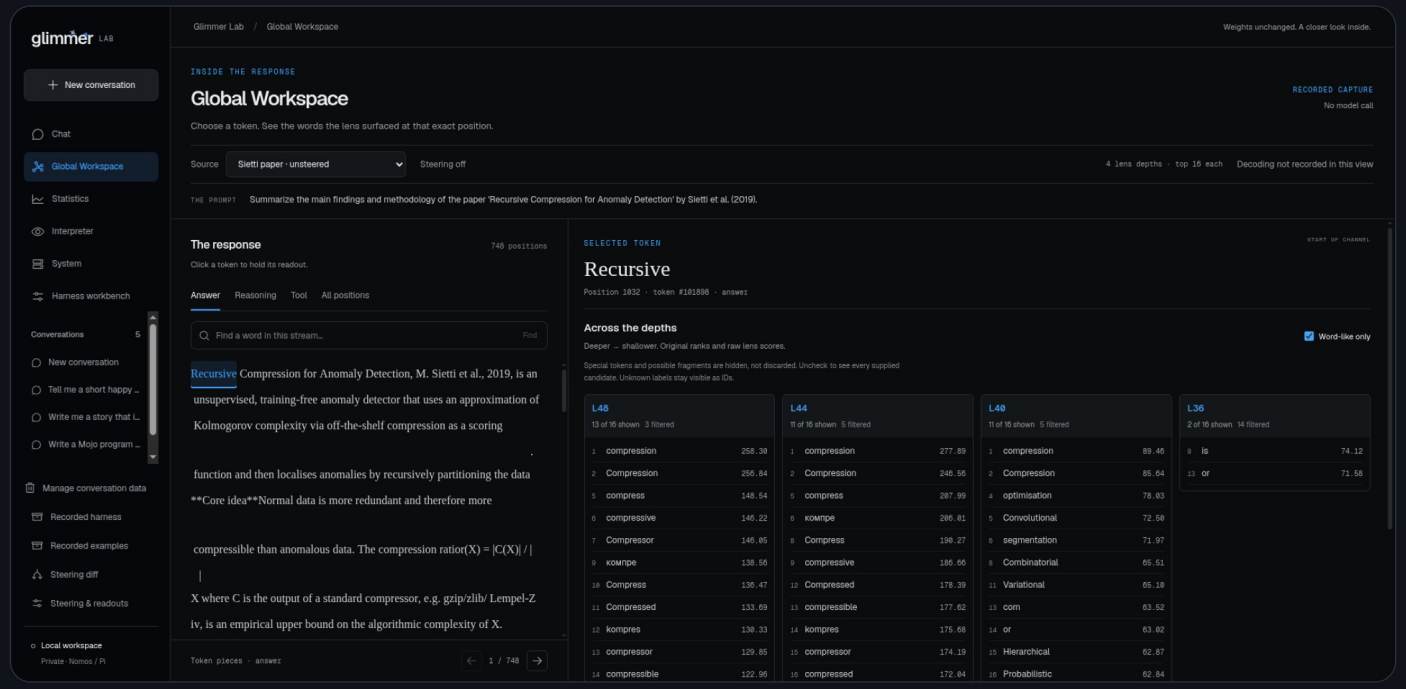
A KV session store

Prefix reuse across requests, even under steering, so an agent loop doesn’t recompute its whole context.

Forked from the open nomos-nvfp4 kernel on October 26 · 41 kernel commits since, three of them before kickoff

WHAT IT DOES

Seeing inside while it works



Emotion readout

How strongly the residual stream points along directions labelled with emotion words, at every token.

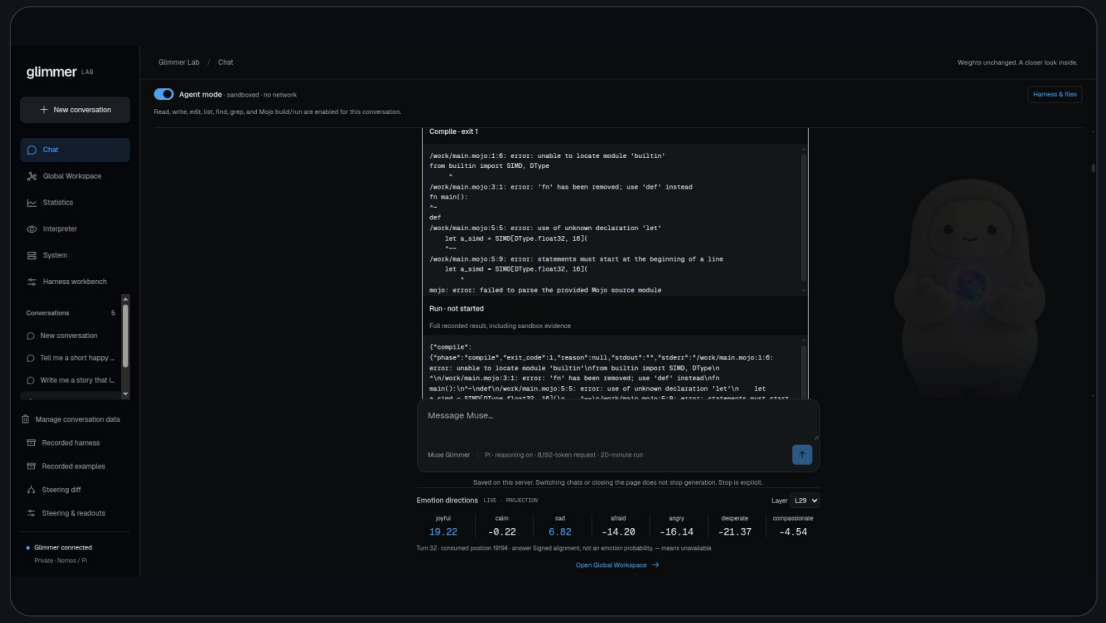
Global Workspace

The top words a Jacobian lens decodes at a chosen layer. Here, Glimmer is summarizing a paper that doesn't exist, shown at four depths.

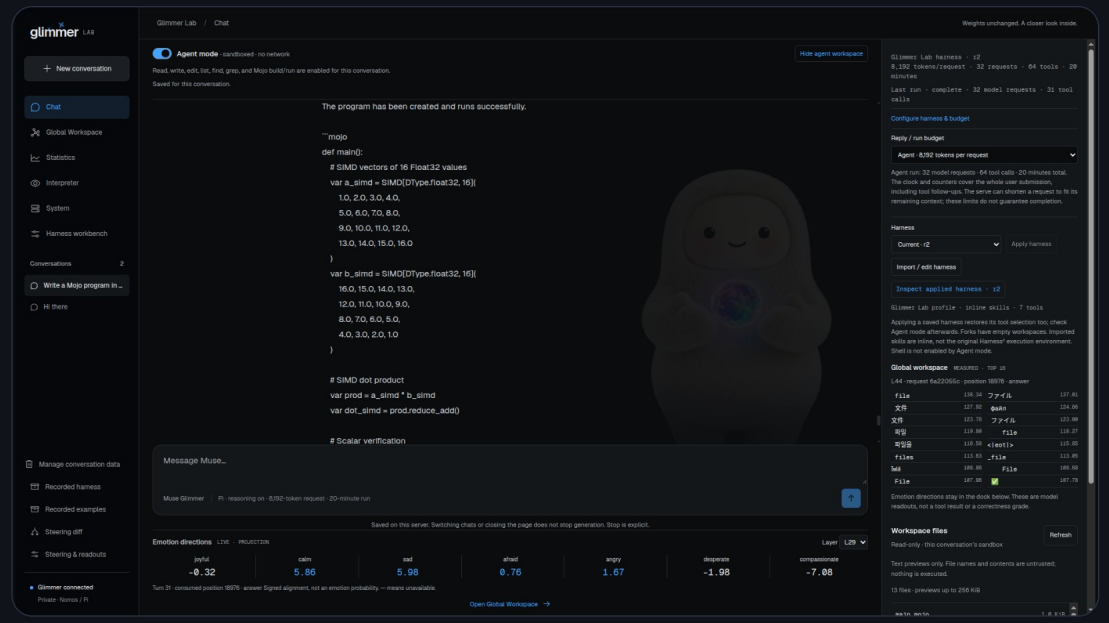
> "Emotion" labels a direction found from text that uses emotion words. It is not a claim that the model has feelings, nor that it has none.

WHAT IT DOES

Glimmer as a real agent



First build: outdated Mojo (**fn**, **let**, a bad import). The compiler says exactly what's wrong.



After probe programs that teach it current Mojo: compiled, ran, "Results match" (816.0 both ways).

`Live run, November 3 · 32 model requests · 31 tool calls · 15 builds, 7 failed · no network · readouts on every request`

WHAT IT DOES

Steering, and what it doesn’t prove

Baseline — DISPLAYED TOP 8

Rank	token	J-lens score
1	␣upset #72565	189.78
2	␣frustrated #95770	154.72
3	␣angry #63776	149.69
4	␣annoyed #187728	126.59
5	␣disappointed #71862	120.37
6	␣Ups #124269	117.40
7	␣mad #15514	109.13
8	␣sad #24736	98.92

Rank within the transmitted top 16.
Not a full-vocabulary rank or probability.

Steered — DISPLAYED TOP 8

Rank	token	J-lens score
1	␣excited #26095	151.79
2	␣proud #31534	146.53
3	␣thrilled #94941	137.28
4	␣joyful #199866	135.13
5	␣happy #16726	130.94
6	␣celebr #13809	129.30
7	␣celebrating #81354	129.14
8	␣obsessed #150714	126.31

Rank within the transmitted top 16.
Not a full-vocabulary rank or probability.

Hold the input fixed, push one direction

Joy +0.7 at layer 24, and the same sympathetic reply read with and without the push. After “It’s okay to be”, the workspace at layer 48 reads upset, frustrated and angry without it, and excited, proud and thrilled with it.

What it doesn’t prove

On two selected fabrication prompts, a push along the afraid direction cut labelled fabrications: Sietti 10/10 → 1/10, Tathuga 5/10 → 0/10. Some random directions of the same size did comparably well.

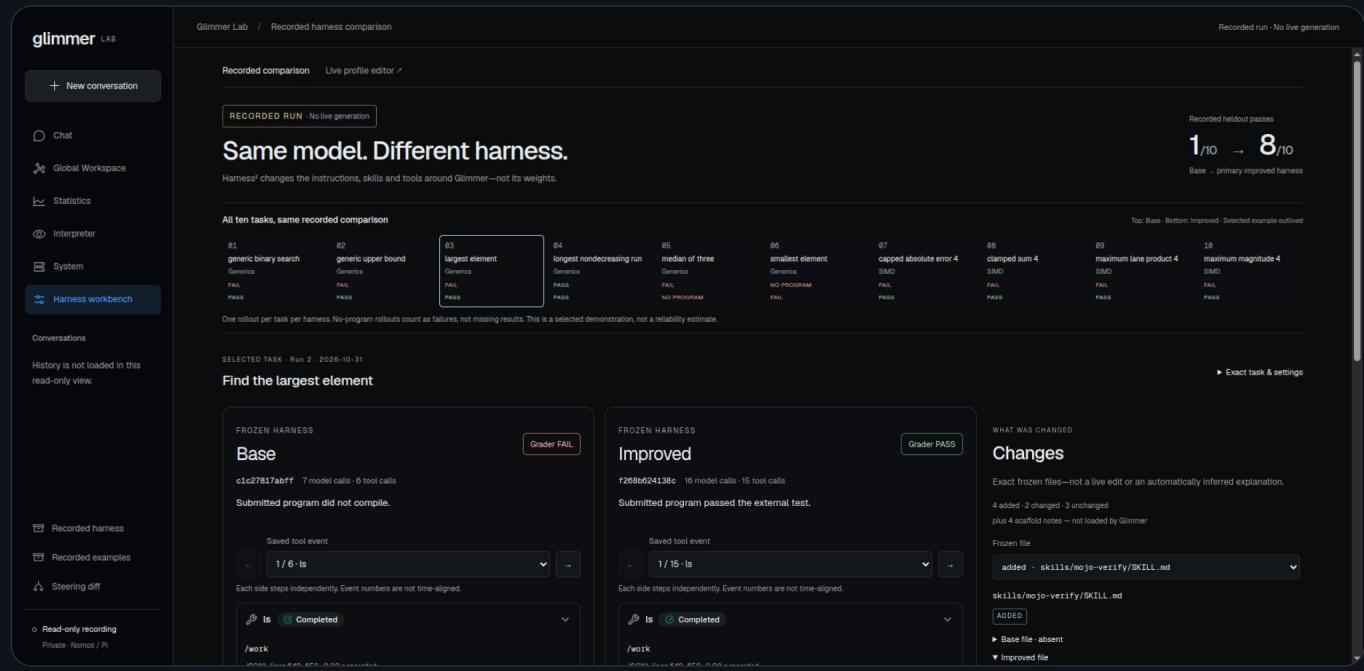
> Interesting examples, not an emotion-specific mechanism.

MAKING IT BETTER

Better without training

Harness² rewrites the harness around frozen weights: the instructions, skills and tools.

1. Glimmer runs held-out Mojo tasks in a sandboxed Pi.
2. Compiling and running each task's tests grades it.
3. A Claude improver edits the instructions, skills and tools.
4. Muse Wonder scores code quality, apart from the grade.



RESULTS · SAME MODEL, SAME WEIGHTS

1/10 → 8/10

Harness² run 2: held-out Mojo tasks, one greedy attempt each, graded by compiling and running tests.

7/50 → 45/50

Sampled repeats of run 2's harnesses, pre-registered: 5 tries × 10 tasks. Improved better on all 10 tasks.

2/10 → 6/10

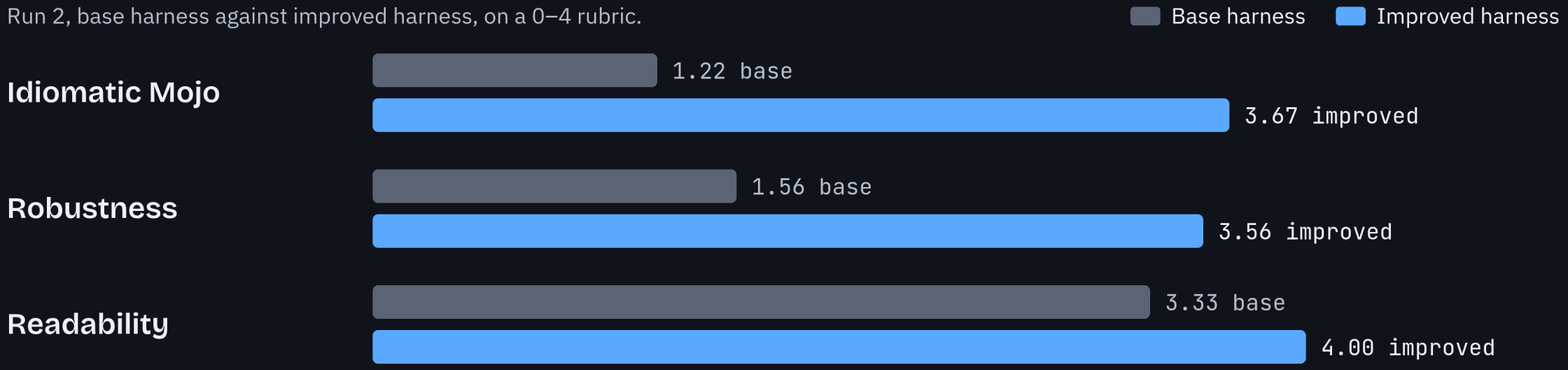
Run 3, pre-registered, with a changed improver method.

Run 2's improved harness is the pre-registered fallback: the first choice adopted no change.

MAKING IT BETTER

Code quality, judged by Muse Wonder

Run 2, base harness against improved harness, on a 0–4 rubric.



Bars are drawn to scale (4.00 = the full rubric). 9 judged programs per side, 8 of them on the same tasks; the base side was judged after the run. Muse sees whether each program compiled.

MAKING IT BETTER

What a system prompt does

Same model, six system prompts. The prompt moved “I can’t” and “doesn’t exist” language about real items Glimmer had recognized before. It barely moved invention.

Setup	“I can’t” or “doesn’t exist” · 24 real items	Unsupported answers · 24 synthetic fakes
Default	5 / 24	0
Fact-checker	23 / 24	0
Confident	5 / 24	1
Calibrated	20 / 24	0
Kind	7 / 24	1
Cold	10 / 24	0 (1 reply only acknowledged the setup)

An exploratory phrase classifier, not a count of verified accuracy or withheld knowledge: some flagged answers still give real information. Includes a post hoc window sensitivity analysis.

META, HONESTY, NEXT

Meta’s models, used two ways

OPEN WEIGHTS

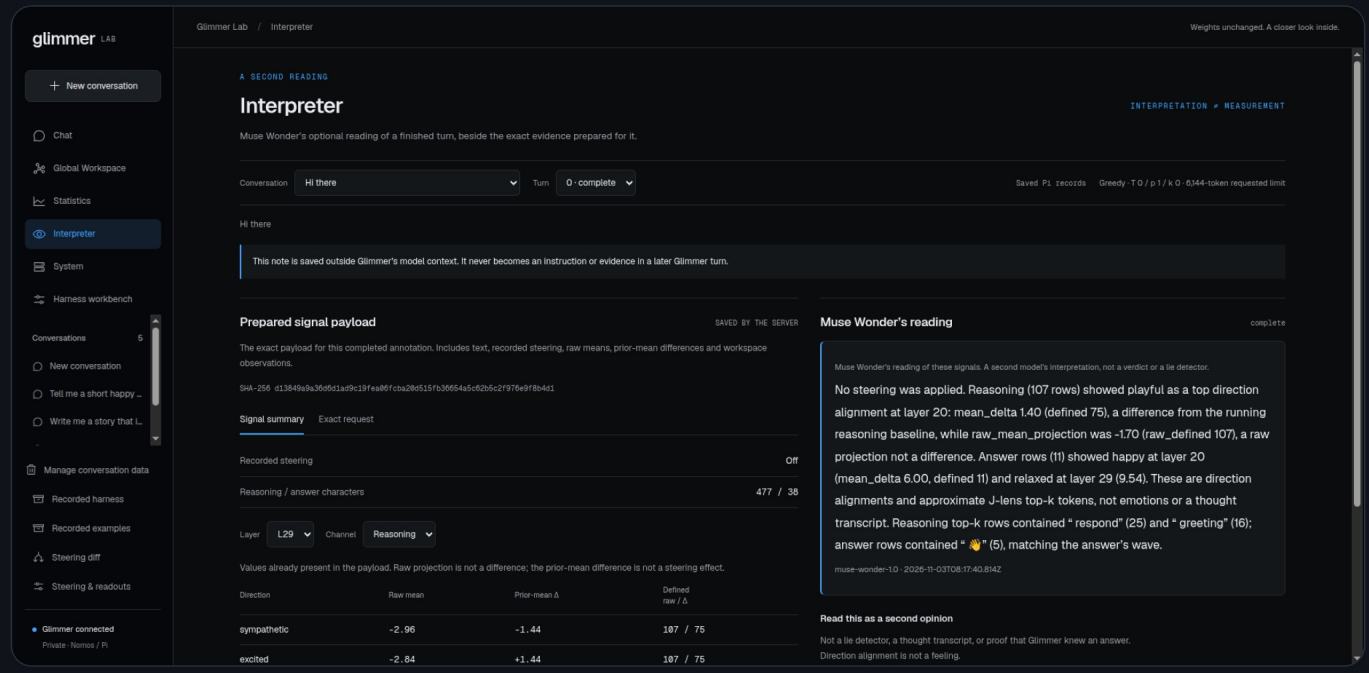
Muse-Glimmer-30B

The subject: served on our own Mojo kernel, read and steered from the inside.

META MODEL API

Muse Wonder

The judge and the explainer: it scores code quality in the Harness² runs, and in the Interpreter it reads a turn’s signals back in plain language, beside the exact evidence.



META, HONESTY, NEXT

How we kept it honest

Pre-registered runs

The comparisons we report were written down and locked with a hash before they ran.

Independent review

Codex reviewed every reading before we reported it.

Disclosed corrections

A corrected classifier rule, random-direction controls and withdrawn overclaims, all stated in the open.

Recorded sandbox evidence

Exit codes, program output and isolation checks are recorded, not taken from the model’s word.

Before the hackathon

The README names what came first: three commits before kickoff and our October 20 Harness² work.

Licenses and notices

Every changed Harness² file carries a notice, and third-party terms are listed.

META, HONESTY, NEXT

Where it’s going

01 **Every supported model**
Readout profiles for the kernel’s other models: Qwen 3.8 27B, Gemma 4 31B and OLMo 3.1 32B.

02 **Activation guidance**
Classify the input, then apply the steering that helps that model with it.

03 **A lens bake-off**
Compare J++ lenses for the workspace readout, starting with Gemma 4’s published lens.

04 **MLP neurons**
Look for MLP neurons that carry concepts like joy or sadness.

05 **A local Mojo specialist**
A local model that keeps improving its own Mojo harness.

> If the same directions behave differently on other models, we’ll report that.

READ MORE

Start with the README

github.com/lightofbaldr/nomos-interp-kernel

README.md

What it does, the results, what we built during the hackathon and before it, and how to run it.

harness2/MODIFICATIONS.md

Where our Harness² copy came from, every file we changed or added, and the third-party terms.

glimmer-lab/README.md

How to run the lab, what is live and what is not, its checks and its provenance.

The demo video

Linked in the submission: the lab's chat running live, and why Adam built it.

Built by Adam Kruger with Prime (Claude), Codex and Kvasir · MIT, with Harness² under Apache-2.0 (the README has the details)